

Team roboticus

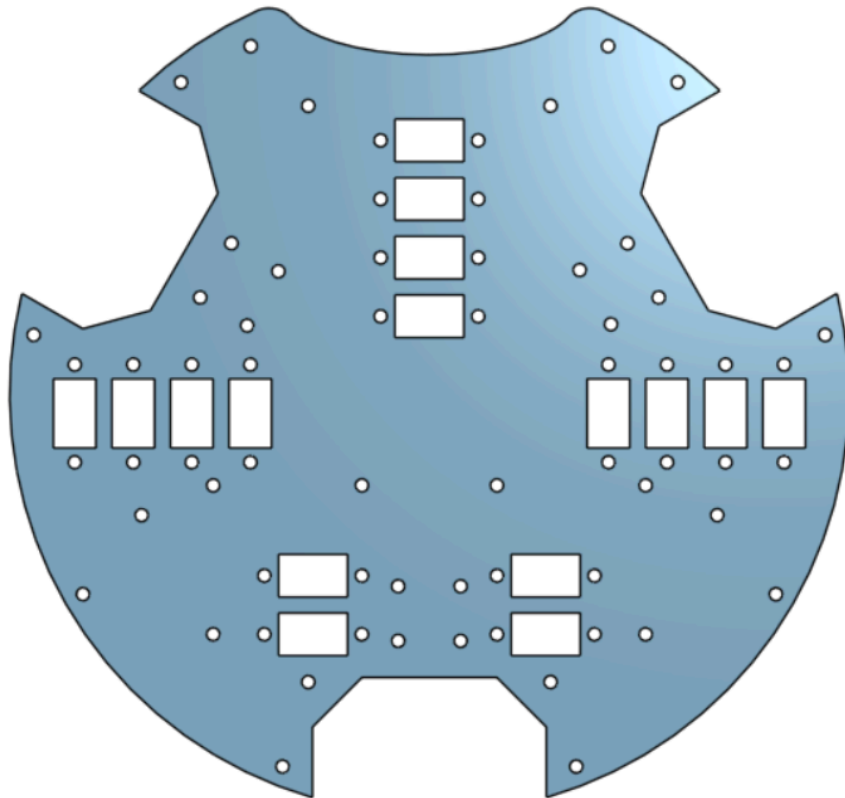
Team roboticus

foto robot

Hardware

foto impressie hardware

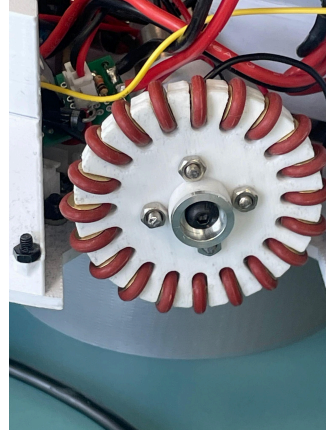
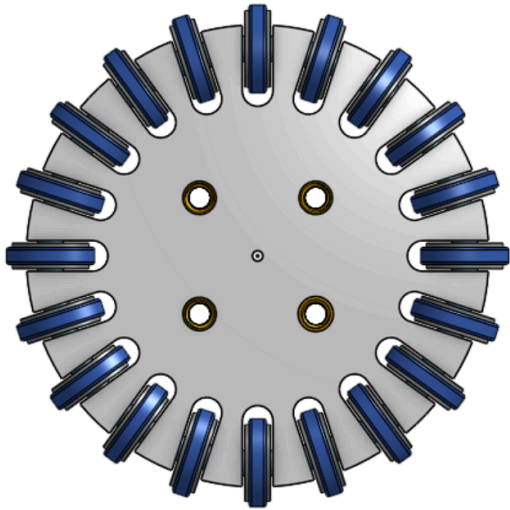
Frame



We have a fully 3D-printed frame which we completely designed ourselves so that all of our parts fit seamlessly on the robot. By creating our own custom frame, we were able to precisely position components such as the motors, electronics, sensors, PCB, and the Arduino Due for the best possible balance and efficiency. This ensures that every part has its own dedicated place, reducing wasted space and making the overall design more compact and organized.

Another advantage of the custom frame is that it improves maintenance and assembly. Since all components are designed to fit perfectly together, wiring is cleaner and replacing parts is much easier. Overall, the custom-designed 3D-printed frame makes the robot stronger, lighter, more efficient, and better suited for our specific application.

Wheels

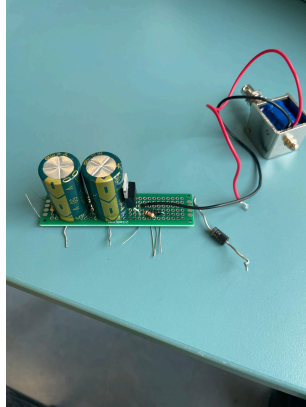


We used a hand designed 60mm omni wheel design. These rubber rollers give the robot much more traction during fast movements and stop it from sliding around. We also spent time refining the fit to make sure the rotation is as smooth as possible, which helps keep the robot stable and reduces vibration when we are driving at high speeds. with these improvements to make the drive system more reliable and responsive.

Before this new omni-wheel we used a simple cheap plastic omni-wheel which we had issues with noise, vibration and grip so this new wheel should fix all of these issues.

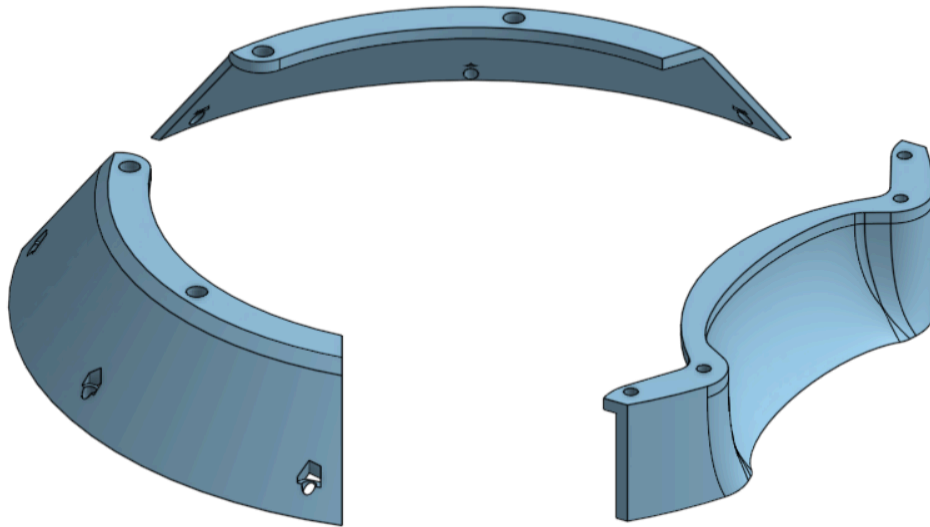
Kicker

We utilize a kicker in our robot which is a solenoid powered by 2 4700 microfarad capacitors that are charged by our 12v battery. We control the solenoid with a transistor connected to the solenoid ground and a digital pin on our arduino due.



Bumpers and capture zone

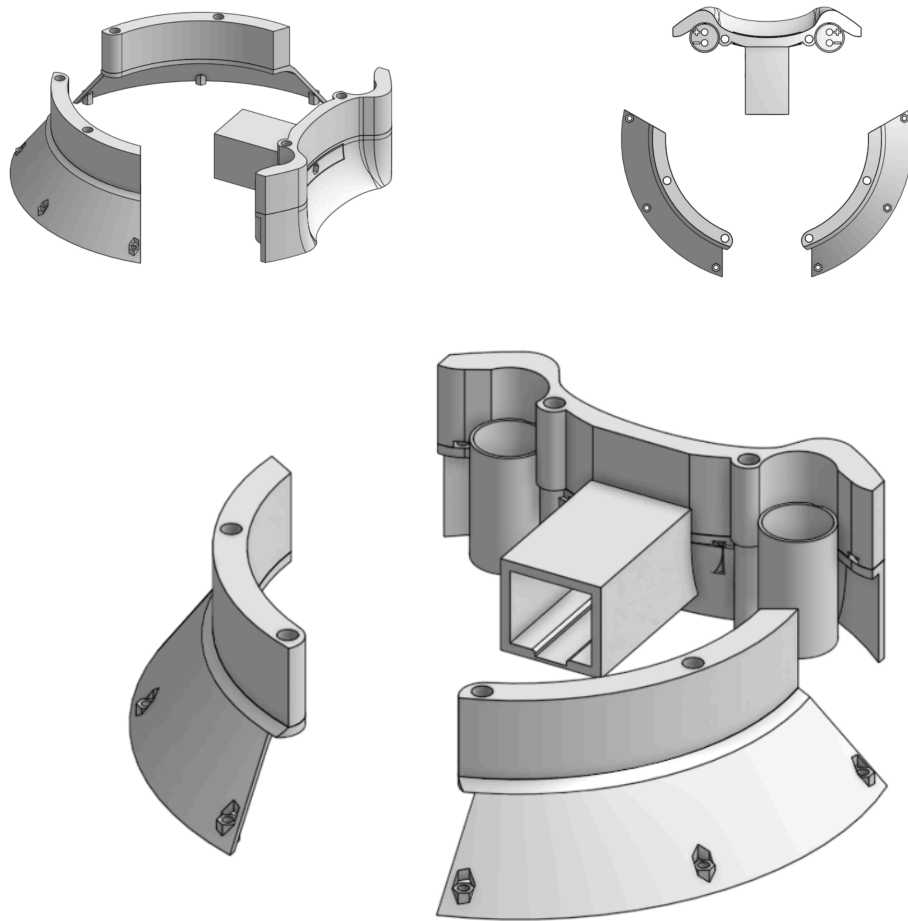
V1



We have 3D-printed bumpers where we integrate the capture zone directly into the bumper structure to minimize weak points and improve the overall strength of the robot. By combining these two parts into a single design, we reduced the number of separate components and mounting points, which lowers the risk of parts breaking or coming loose during impacts or fast movements. This also makes the robot lighter and more compact while keeping the structure strong and reliable.

The capture zone is designed in a way that it almost grabs and guides the ball automatically. Because of the curved shape and carefully designed angles, the ball is directed toward the center of the robot when it makes contact with the bumper. This improves ball control and makes it easier for the robot to secure the ball quickly, even when approaching at different angles or higher speeds. The integrated design also helps absorb impacts and keeps the ball from bouncing away too easily. As a result, the robot can collect and control balls more consistently and efficiently during operation.

V2



We designed and 3D-printed custom bumpers in which the capture zone is fully integrated into the bumper structure itself. By combining these functions into a single component, we reduced the number of separate parts and mounting points, making the overall construction stronger, lighter, and more reliable. Fewer individual components also lower the risk of parts shifting, loosening, or breaking during impacts or fast movements on the field. In addition, the integrated design makes the robot more compact and easier to assemble and maintain.

The capture zone was specifically shaped to improve ball intake and control. Thanks to the curved geometry and carefully optimized angles, balls are naturally guided toward the center of the robot when they come into contact with the bumper. This creates an almost self-centering effect, allowing the robot to secure balls more quickly and consistently, even when approaching from different directions or at higher speeds.

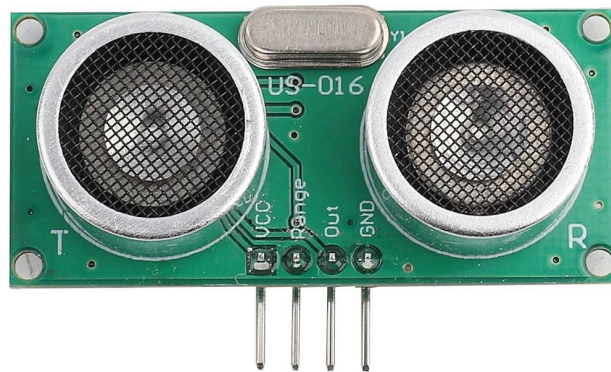
Behind the capture zone, we integrated the housing for the kicker mechanism. The rectangular structure located behind the bumper supports and protects the kicker system, while also keeping it aligned with the center of the robot for accurate ball launching. The small front bar acts as the actual kicking element: once activated, it rapidly pushes the ball forward to shoot or pass it with force and precision.

The cylindrical tubes marked with positive and negative symbols are designed to hold the capacitors that power the kicker system. These capacitors can store a large amount of electrical energy and release it instantly when the kicker is activated, providing the high power needed for fast and powerful shots.

The integrated structure also helps absorb and distribute impact forces more effectively, preventing the ball from bouncing away too easily. As a result, the robot can collect, guide, control, and shoot balls in a smoother and more efficient way during operation, improving both reliability and overall gameplay performance.

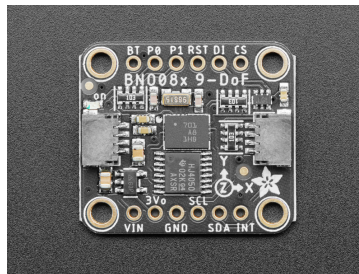
Sensors

Ultrasonic sensors

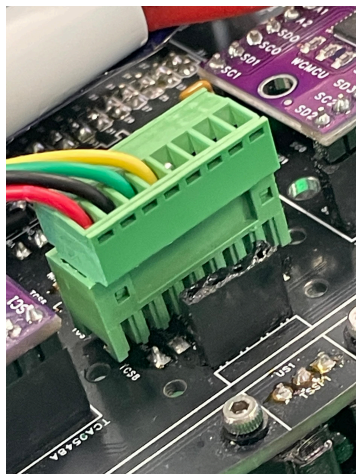


We use four US-016 ultrasonic sensors to help the robot detect its distance from the walls. These sensors are positioned to give us constant feedback on the robots position so we can prevent collisions and stay within the field boundaries. We chose the US-016 model specifically because it provides an analog output, which makes it much easier to get fast and stable distance readings compared to standard digital versions. This allows the robot to make quick adjustments while driving without any lag. By integrating these sensors directly into our code, we can keep the robot centered and ensure it always knows exactly how much space it has to maneuver during a match.

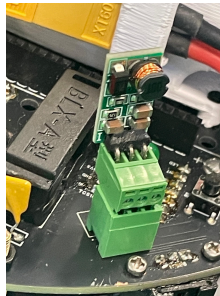
BNO085



To keep the robot facing the right way we used a BNO085 orientation sensor connected over I2C. This sensor is basically the brain for our heading and PID control. We went with this specific model because it does all the heavy lifting internally. It combines the gyro and accelerometer data itself so we get a clean stable angle without our main code having to do a ton of math. This steady data is what lets our PID controller work smoothly making those quick little motor adjustments to keep the robot on track or snap it to a new heading instantly. It is a reliable way to make sure the robot does not lose its sense of direction during the middle of a game. We have put a 3d printed shell over the bno to protect it from external forces and shorts. Additionally we added screw terminal blocks in order to easily replace the bno if something goes wrong.



3.3V Regulator



Our 3.3V regulator is one of the most important pieces of hardware on the robot, this voltage regulator turns our 12v battery voltage into a useable 3.3v for almost every other component. Without this almost everything wouldn't work. Similar to the BNO085 we put screw terminal blocks so we can change it quickly without having to solder anything saving time and stress

TSSP4038



To find the ball, we used TSSP4038 sensors to pick up the specific IR signature from the soccer ball. These sensors are great because they are tuned to a 38kHz frequency, which means they are really good at filtering out background noise and other light sources that might interfere during a match. We placed them around the robot with fov limiting caps to give us a full 360 degree view of the field. The caps placed on them so the sensors don't overlap on each other which can cause confusion for the robot. By constantly checking the signal strength on each sensor, the robot can figure out exactly where the ball is and head straight for it. It is a simple and effective way to make sure we are always locked onto the IR signal without the code getting confused by reflections or overhead lights.

TCS3472

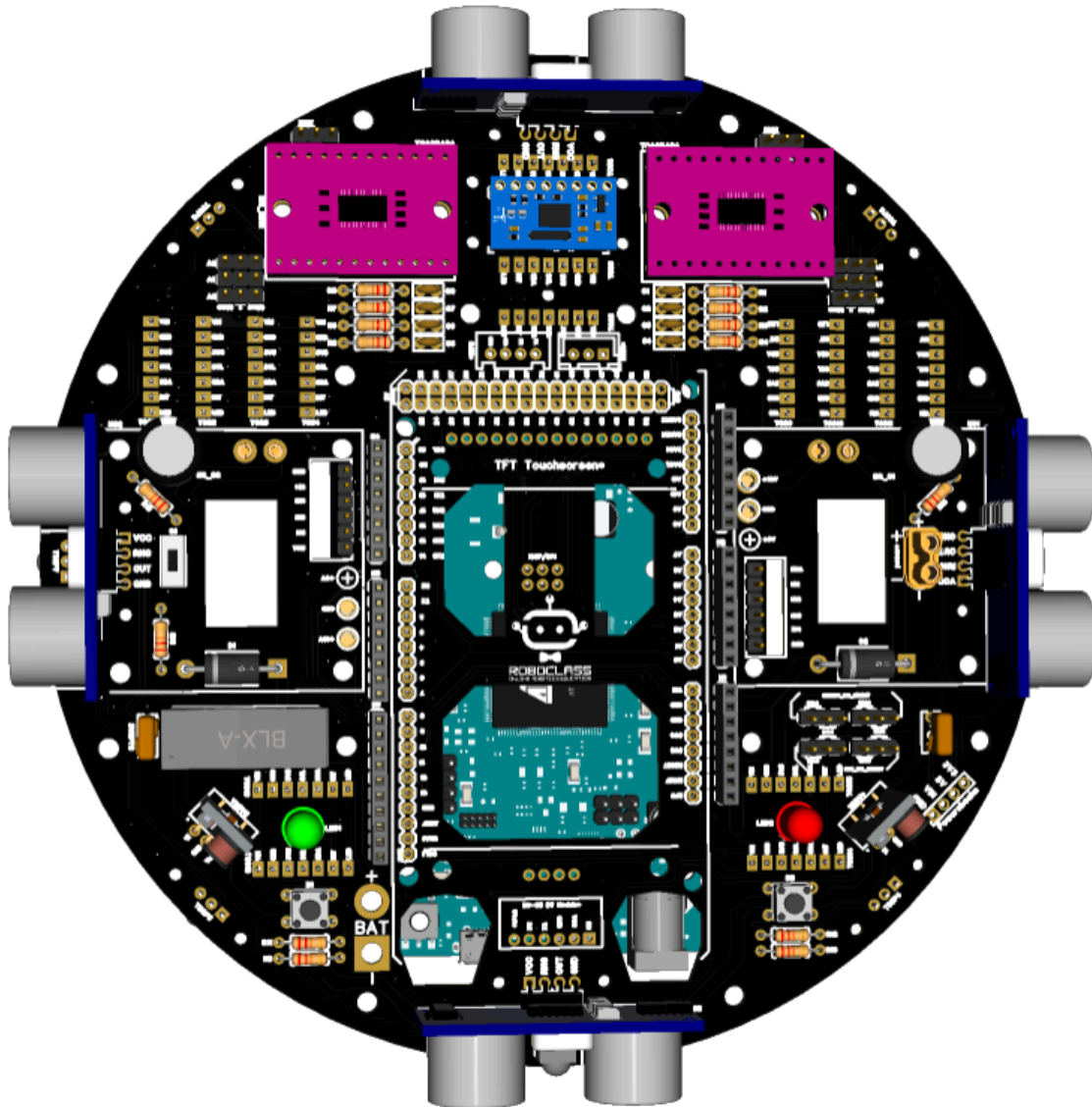


For staying within the field boundaries, we have a TCS3472 color sensor that communicates over I2C. This sensor is positioned facing the floor so it can constantly scan the surface colors as the robot moves. We use it specifically to detect the white lines that mark the edge of the pitch. Because it can distinguish between the green of the mat and the white of the boundary lines, we can program the robot to immediately stop or back up the moment it senses a change in color. Using I2C makes it easy to integrate with our existing setup, giving us a reliable way to make sure the robot never accidentally drives out of bounds during a match.

TCA9548A

Since we have a lot of color sensors to handle the field boundaries, we had to use a TCA9548A I2C multiplexer. The main issue is that most color sensors have the same fixed I2C address, so you cannot just plug sixteen of them into the same bus without a conflict. This multiplexer acts like a switchboard, letting the main controller talk to one sensor at a time by switching between different channels. It keeps the wiring much cleaner and allows us to poll each sensor individually to get a full map of the floor. Without it, we would never be able to run that many sensors on a single I2C line, so it is really the part that makes our boundary detection system possible.

PCB

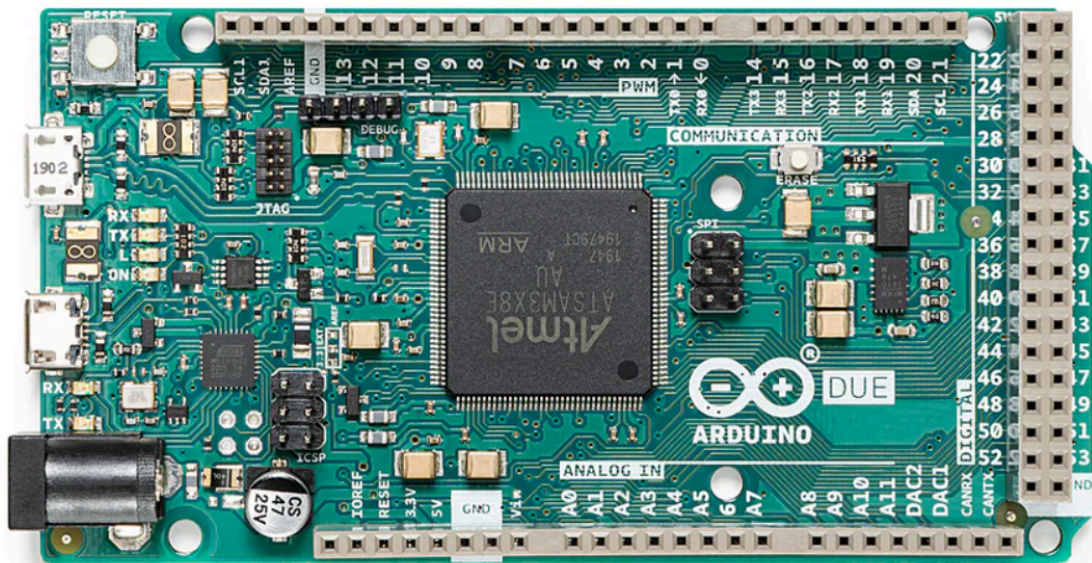


We use a PCB (Printed Circuit Board) in our robot to create a reliable and organized electronic system. A PCB allows all electronic components, such as sensors and the Arduino Due, to be connected neatly and securely. This reduces loose wires and makes the robot more compact and professional.

Another advantage of using a PCB is that it improves reliability. Since the connections are permanently built into the board, there is a lower chance of wires coming loose or causing electrical problems during operation. This is especially important in a moving robot where vibrations and movement can affect normal wiring.

A PCB also makes troubleshooting and maintenance easier because the circuit layout is clear and well-structured. In addition, it helps save space and can improve electrical performance by reducing interference and resistance in the connections. Overall, using a PCB makes the robot more durable, efficient, and easier to work with.

Arduino Due



We use the Arduino Due in our robot because it is fast, powerful, and capable of handling multiple tasks at the same time. The board uses a 32-bit ARM processor running at 84 MHz, which gives much more processing power compared to standard Arduino boards like the Uno. This allows our robot to process sensor data quickly and react faster to its environment.

Another reason we chose the Arduino Due is the large number of input and output pins. These pins make it possible to connect motors, sensors, communication modules, and other electronic components easily. The board also supports advanced communication methods such as SPI, I2C, and UART, which are useful for connecting different systems inside the robot.

The Arduino Due is also suitable for projects that require precise control and real-time calculations. Because of its higher memory capacity and faster processor, it can run more complex programs without performance problems. This makes it a reliable and efficient choice for our robot project.

Software

General

Development environment

Our code is written completely in C++ and uploaded via the Arduino IDE using the programming port on our Arduino DUE. We chose this particular software for its high clock speed, multiple hardware serial ports and compatibility with our arduino microcontroller, which are essential for fast uploading and real time processing.

Libraries and communication

our software relies on several libraries to handle complex sensor communication:

- **Wire.h:** Used for I2C communication between the sensors and the microcontroller.
- **QuickPID:** An optimized library for our PID controller to ensure fast motor corrections.
- **Adafruit BNO085 & TCS34725:** Libraries used to simplify the data acquisition from the compass and color sensors.

Program flow

the code starts with defining all the variables, sensors and functions, after that it follows a continuous loop of sensing, thinking and acting

1. **sensing:** the robot reads out all IR, Ultrasonic and I2C sensors at the start of every loop.
2. **processing:** the raw data gets converted using threshold and the PID algorithm. this is where the robot determines the ball's location and its own orientation.
3. **acting:** based on the processed data, the drive() function calculates the speed of each of the three motors to move in the desired direction.

Code hierarchy

to keep the code organized and efficient, we use a priority based system:

High priority: scoring and maneuvering the ball once it's captured.

Medium priority: tracking the ball using IR sensors.

low priority: failsafe search patterns when no ball is detected.

picture impression software

Drive-base

when we had to redesign our robot to fit three wheels instead of four, we also had to completely remake the drive-base and PID, while doing this we found that in most directions the wheels need to be either idle or moving at full speed but when driving to the left or the right we needed all three wheels but the two in front of the robot need to move slightly slower

than the back wheel so that it doesn't turn but instead drives steadily into the direction it needs to to capture the ball. We also found that we don't need to perfectly create the directions and wheel speeds because our PID does most of the work with driving perfectly straight since it adjusts the speeds of each wheel to account for minor imperfections keeping the robot straight.

picture drive-base

PID

We use a proportional-integral-derivative(PID) controller to make sure the robot will always be facing to the opponents side unless instructed to turn to the goal of the opponent. we found that if we don't use this the robot will most of the time have a really curved path instead of driving straight because not every motor reacts the same when given the same instructions so we had to implement this feature to account for these minor imperfections in the motors. This protocol uses our compass to detect where the robot is facing, this sensor sends out a signal of 0 to 359 depending on how it turned since turning it on. The PID converts this signal to a signal from -180 to 180 so that it knows if it needs to adjust to the left or to the right to face straight again.

```
bno.setSensorOffsets(calibrationData);
delay(10);
beginHeading = event.orientation.x;
Serial.print("START-VALUE: ");
Serial.println(beginHeading);

bno.getEvent(&event);
float rawInput = event.orientation.x;
float adjustedInput = rawInput - beginHeading;
if (adjustedInput > 180) {
    adjustedInput -= 360;
} else if (adjustedInput < -180) {
    adjustedInput += 360;
}
Input = adjustedInput;
myPID.Compute();
OutputPID = (int)Output;
```

I²C

We use 2 Inter-Integrated-Circuit(I²C) addresses to control both the color sensors(TCS3472) and the compass(BNO085). for the color sensors which we mostly used in the Dutch national championship to detect the lines at the edges of the field we decided that if it detects a line it should move in the opposite direction of where it detects the line to always stay in bounds however, for this competition we got rid of these sensors because we don't

need them and they were really annoying when we have to take apart the robot and unplug every single cable of these sensors.

The compass still has the same use as in the Dutch national championship, to send data to the PID so that it can always keep driving straight. We reset this compass in the setup so that it will always be using the direction in which it started when you turn on the robot.

To make sure that our microcontroller doesn't mix up those sensors, we use two multiplexers to communicate with multiple devices on the same bus to prevent address conflicts between identical sensors using the addresses 0X70 and 0X71.

```
#define MUX1_ADDR 0x70  
#define MUX2_ADDR 0x71
```

Sensor Data

General

Most of our sensors send an analog signal from 0 to 1023, unlike our color sensors which send a signal up to 65.535 since it uses a 16-bit resolution.

IR sensors

For our IR sensors(tss4038) we use multiple thresholds so the robot can drive to the ball and then go around it to get the ball in the capture zone. Every sensor uses either two or three thresholds. The first one is to see if it detects that the ball is near that sensor, this one is the same for every one 1005, we chose this specific threshold because it is low enough to see if it detects the ball and high enough to make sure that if the ball gets detected that the robot will know that it got detected. The rest of the thresholds are different for every sensor but most of them use the same one as the sensor in the opposite position. Using those sensors we can accurately predict where the ball is and how we need to drive to get to the ball, the lower the signal the closer the ball. Our thresholds are pretty low because we want the circle in which the robot drives around the ball to be as small as possible so that the risk of colliding with a wall or goal is as low as possible.

Ultrasonic sensors

For our ultrasonic sensors(US-016) we use two thresholds to define three zones on the field so we can accurately predict where the goal is and how the robot needs to drive to get there. It only activates these zones once the ball has been captured in the capture zone in front of the robot. It also uses these zones to predict how it needs to turn to face the goal when it has the ball so it can drive straight to the goal instead of having the risk of losing the ball when driving diagonally.

Behaviour hierarchy

First priority

Our primary objective is driving and turning towards the goal once it has captured the ball. it detects when it has captured the ball using 2 microswitches in the capture zone, when it is really close it will assume it has the ball and turn to the goal in an approximately 35 to 45 degree angle using the Setpoint function of our PID which can temporarily change the zero point where it corrects it's heading to.

Second priority

Our second priority is driving towards and around the ball. when the signal is above a certain threshold it will drive straight at the ball until the signal gets below that threshold. When it's lower than the threshold it will drive around the ball until it is behind it. When the ball is in front of the robot it will drive straight to capture the ball in its capture zone.

```
if (ballCaptured){  
  if(US_value2 > US_value4){  
    drive(0,(speed + 10),OutputPID);  
  }else if(zone == 2){  
    drive(0,(speed+ 10),OutputPID);  
  }else if(zone == 3){  
    drive(0, (speed + 10),OutputPID);  
  }else{  
    drive(135,speed,OutputPID);  
  }  
}
```

Third priority

our third and last priority is driving towards the middle of the field once the ball isn't detected hoping that it can pick up the signal again while driving there, it figures out where the middle is by looking in which zone it is and how it need to drive to get to the middle zone, when it hasn't detected the ball when its in the middle, it will drive in a random direction to search for the ball.

Failsafe mode

to prevent the robot from becoming stationary when losing track of the ball and giving the opponent a free goal, we implemented a failsafe search pattern. If all IR sensors (TSSP4038) return a value above the detection threshold, the robot enters a default search state. In this state, it utilizes the ultrasonic zones to navigate back to the center of the field. This ensures the robot remains active and maximizes its chances of re-acquiring the ball's infrared signal rather than idling in a corner.

Kicker

We use a kicker mechanism to kick the ball towards the goal, we do so by turning the robot approximately 35 to 45 degrees towards the goal. The robot detects if the ball is captured in the capture zone by using 2 microswitches in the capture zone. If one of them sends a signal to the arduino it will know when it has captured the ball. Once the ball is captured, the robot detects where the goal is by using the ultrasonic sensors to detect which wall is closest so it knows whether to turn to the left or to the right. Once the robot is turned towards the goal it looks if the goal is close enough to be able to shoot the ball towards the goal. If it is close enough it will send a digital HIGH signal to the solenoid for about 200 milliseconds so the charge is released and the solenoid will extend shooting the ball straight into the goal.